

ANUBHAV PANDEY

New Delhi, India • anubhavp950@gmail.com • +91 97594 10300 • github.com/anbv29 • linkedin.com/in/anbvp • portf-chi-two.vercel.app

SUMMARY

Final-year B.Tech student at NIT Jalandhar with hands-on experience building full-stack and backend systems on the PERN stack (PostgreSQL, Express.js, React.js, Node.js) and Next.js/serverless architectures. Comfortable across RESTful API design, concurrency-safe backend systems, secure payment integrations, and AI features built with Retrieval-Augmented Generation (RAG), LLMs, and vector search. Looking for Software Engineer / Software Developer roles.

TECHNICAL SKILLS

Languages: Java, Python, JavaScript, TypeScript, SQL

Frontend: React.js, Next.js

Backend: Node.js, Express.js, Fastify, FastAPI

Databases & Caching: PostgreSQL, Redis, Neon DB, Supabase, pgvector, Prisma (ORM)

AI/ML: RAG, LLMs, Multi-Agent Systems, Embeddings, Semantic Search, OCR

Tools & Platforms: Git, Docker, Terraform, BullMQ, Vercel, GitHub Actions (CI/CD)

Concepts: RESTful API Design, System Design, Auth (JWT, RBAC, OAuth), Async Job Processing, Concurrency Control, Rate Limiting, Row-Level Security (RLS), Payment Gateway Integration, Vector Search

EXPERIENCE

Software Engineering Intern — Reach Inc.

Jun 2025 – Jul 2025

- Built backend logic and REST API endpoints in Node.js and Express.js for new web application features, taking each from spec through deployment.
- Partnered with the engineering team across the full development lifecycle — design discussions, implementation, testing, and release.
- Wrote server-side integrations for core site functionality, following the team's existing REST API design conventions and best practices.

PROJECTS

Throttle — React.js, Node.js, Express.js, PostgreSQL (PERN Stack)

- Built a standalone rate-limiting service using Token Bucket and Sliding Window algorithms, backed by PostgreSQL ACID transactions and row-level locking (SELECT ... FOR UPDATE) to keep ALLOW/DENY decisions correct under load — held up through tests simulating 1,000+ concurrent requests/sec with zero race-condition errors.
- Added a real-time analytics dashboard and packaged the limiter as drop-in middleware, cutting integration time for a second test service from a full day to under an hour.

Finora — React, TypeScript, Node.js, Fastify, Python, FastAPI, PostgreSQL, pgvector, Redis, BullMQ, Docker

- Built a multi-tenant document intelligence platform with async PDF ingestion, OCR, document classification, and schema-driven structured data extraction — processed 500+ test documents at over 90% field-extraction accuracy.
- Added a RAG pipeline (chunking, embeddings, pgvector, semantic search) for grounded, cited LLM answers, cutting incorrect or irrelevant responses by roughly 40% versus a no-retrieval baseline; secured the API with JWT auth, refresh-token rotation, RBAC, and tenant isolation, shipped as Dockerized services.

MYDAY — Next.js, TypeScript, PostgreSQL (Supabase), Clerk, Razorpay, Upstash Redis

- Built a full-stack platform for claiming and monetizing calendar dates, pairing Clerk auth with Supabase Row-Level Security so authorization is enforced server-side, plus a concurrency-safe bidding system (partial unique indexes, webhook-verified idempotent transactions) that held to exactly one valid claim per date across simulated bursts of 200+ concurrent bids.
- Wired up dual-currency (INR/USD) checkout through Razorpay with signature-verified webhooks and fail-closed logic, added Upstash Redis rate limiting, and shipped CI/CD to Vercel via GitHub Actions.

Precedent — Python, LLM Agents (technical interview exercise)

- Built a two-stage agent system that derives a law firm's negotiation playbook from its past contracts, then reviews new drafts clause-by-clause against it.
- Classified every clause as accept, counter, or escalate with citations back to the source documents, and made reviews deterministic — validated against a held-out corpus from a different firm.

Wajo — Python, LLM Agents, CLI (technical interview exercise)

- Built an agent loop for proactive email handling — observe, normalize, assess risk, interpret, choose autonomy, act or wait, learn — using an LLM as the semantic planner with no direct mailbox access.
- Kept a deterministic Python policy layer in control of every action, binding approvals to exact payloads, and let autonomy increase over time only for actions that were both safe and reversible.

EDUCATION

National Institute of Technology (NIT), Jalandhar — B.Tech

Nov 2022 – Jun 2026